Functions are values

A function is an object with a name, so it can get a second name, sit in a list, and be passed to another function.

THE QUESTION

What can you do with a function besides call it?

THE INTUITION

`def` makes an object and points a name at it, as `x = 5` does. So a function can get a second name, sit in a list, or go into another function that calls it later.

KEY TAKEAWAY

`f` is the function; `f(x)` is what it returns. Hand over `f` without brackets to `map`, `filter`, `key=` or your own `apply_twice`.

`def square(x): ...` → a function object → `square`: a name for it → `apply_twice(square, 3)` → `f(f(3))` inside → 81

1 `square` is the function; `square(4)` calls it

fn.py

```
1 def square(x):
2     return x * x
3 print(square)
4 print(type(square))
5 print(square(4))
```

output

```
<function square at ...>
<class 'function'>
16
```

`square` → the function itself, a value: store it, pass it on

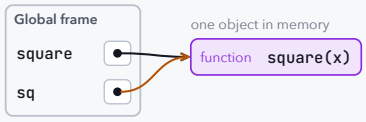
`square(4)` → a call: run the body with x = 4, and get back 16

`def` (the functions sheet) makes a **function object** and points `square` at it. The bare name is that object, a value like 5. Where we print `...`, Python shows the object's **memory address**, a number that changes every run. Round brackets **call** it. Built-ins are objects too: `print(len)` shows `<built-in function len>`.

2 Two names can point at one function

alias.py

```
1 def square(x):
2     return x * x
3 sq = square
4 print(sq(5))
5 print(sq is square)
6 print(sq)
```



after line 3 (amber: the arrow it drew)

prints `25 True <function square at ...>`

the object keeps its own name: `sq` prints as `square`

`sq = square` copies no code: it adds a second arrow to the same object, as `b = a` does for a list (the names and objects sheet). So `sq(5)` runs `square`'s body. The notebook renames a built-in: after `say = print`, `say("Hello", 42)` prints `Hello 42`.

3 Functions can sit in a list or a dictionary

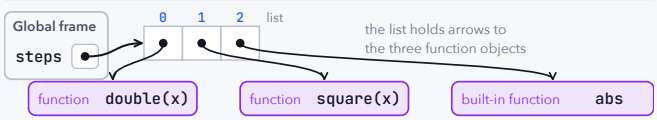
double and square as on this page

```
steps = [double, square, abs]
for f in steps:
    print(f.__name__, f(-3))
calc = {"+": add, "*": mul}
print(calc["*"](6, 7)) → 42
```

the loop, pass by pass

pass	f is	f(-3)	printed
1	double	-6	double -6
2	square	9	square 9
3	abs	3	abs 3

f(-3) calls whichever one f points at



A list holds arrows to any objects, functions too. In the loop, `f` points at each function in turn and `f(-3)` calls it; `f.__name__` is its name. A **dictionary** (the dictionaries sheet) finds a value by a **key**: `calc["*"]` looks up `mul`, and `(6, 7)` calls it.

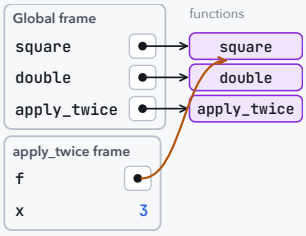
4 Pass a function in; it gets called inside

twice.py (square, double as before)

```
1 def apply_twice(f, x):
2     return f(f(x))
3
4 print(apply_twice(square, 3)) → 81
5 print(apply_twice(double, 3)) → 12
```

inside `apply_twice(square, 3)`:

```
f(x)      → square(3) = 9
f(f(x))   → square(9) = 81
```



amber: f, a second arrow to square

A function that takes or returns a function is a **higher-order function**. In the call's **frame** (the functions sheet) the parameter `f` is one more name for `square`, so `f(f(x))` is `square(square(3))`. With `double` the same code gives 12; `apply_twice(double, "ab")` gives `abababab`.

5 Write your own map: `do_to_all(f, items)`

do_to_all.py

```
1 def do_to_all(f, items):
2     out = []
3     for item in items:
4         out.append(f(item))
5     return out
```

`do_to_all(square, [1, 2, 3])`

pass	item	f(item)	out after
1	1	1	[1]
2	2	4	[1, 4]
3	3	9	[1, 4, 9]

```
do_to_all(square, [1, 2, 3])      → [1, 4, 9]
do_to_all(len, ["hi", "hey", "a"]) → [2, 3, 1]
do_to_all(int, ["7", "42"])       → [7, 42]
```

One loop, any job: the loop builds the new list and `f` says what to do to each item. Any one-argument function fits: your `square`, the built-in `len`, or `int` to turn text into numbers. Python has this loop built in as `map` (page 2).

6 Newton's method takes the equation as an argument

the notebook's newton1

```
1 def newton1(f, df, g0):
2     while True:
3         g1 = g0 - f(g0)/df(g0)
4         if abs(g1-g0) < 0.001:
5             break
6         g0 = g1
7     return g1
```

`g1 = g0 - f(g0) / df(g0)`

`sqrt2(x) = x*x - 2, dsqrt2(x) = 2*x`

pass	g0	g1	|g1-g0|
1	1.1000000	1.4590909	0.3590909
2	1.4590909	1.4149037	0.0441872
3	1.4149037	1.4142137	0.0006900

rounded to 7 places; green: under 0.001, stop

roots 2 and -3

`f(x) = x*x + x - 6, df(x) = 2*x + 1`

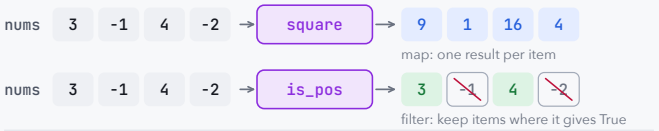
```
newton1(f, df, 3)   → 2.0000000000017724
newton1(f, df, -5)  → -3.0000000098600994
```

A **root** of f is an x with $f(x) = 0$. **Newton's method** improves a guess `g0` again and again (an **iterative method**) until two guesses differ by under 0.001. Written once, it takes f and its derivative `df` as arguments: `sqrt2` finds $\sqrt{2}$. From -0.5, `df(g0)` is 0: a **ZeroDivisionError**.

Hand your function to Python's tools

map and filter, sorting and picking with key=, lambda for one-line functions, and a function that makes functions.

7 map changes every item; filter keeps some




```
print(map(square, nums))           → <map object at ...>
print(list(map(square, nums)))     → [9, 1, 16, 4]
print(list(filter(is_pos, nums))) → [3, 4]
sum(map(int, "3 14 15".split()))  → 32
```

`map(f, items)` calls `f` on every item; `filter(test, items)` keeps the items where `test` gives True. Both return an **iterator**: it works out each result only when asked, and prints as `<map object at ...>`. Use `list()` or a loop. It runs out after one pass: a second `list(m)` gives `[]` (the iterables sheet).

8 key= says what to compare

```
words = ["banana", "fig", "apple", "kiwi", "cherry"]
def last(w): return w[-1]
sorted(words, key=len)
```

fig	kiwi	apple	banana	cherry
3	4	5	6	6

key — banana and cherry tie on length: they keep their first order

```
sorted(words, key=last)
```

banana	apple	fig	kiwi	cherry
'a'	'e'	'g'	'i'	'y'

key

`sorted(words, key=len)` calls `len` on each word and orders the **words** by the results, the **keys**. Pass the function uncalled (`key=` is a **keyword argument**, see the iterables sheet). Ties keep their order: `max(words, key=len)` is banana, `min` is `fig`. `reverse=True` flips the order.

9 lambda writes a small function in one line

these make the same function:

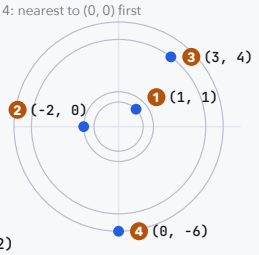
```
def square(x):
    return x * x
```

```
square = lambda x: x * x
```

x: the parameters x * x: one expression, returned

```
square(6) → 36
print(square)
→ <function <lambda> at ...>
sorted(pts, key=lambda p: p[0]**2 + p[1]**2)
→ [(1, 1), (-2, 0), (3, 4), (0, -6)]
```

1 to 4: nearest to (0, 0) first



`lambda x: x * x` is a function with no name of its own: parameters before the colon, one **expression** after it, whose value is returned. It suits a short one-off key: $x^2 + y^2$ orders points as distance does. Anything longer, or reused, deserves a `def`.

10 A function can return a function

the notebook's sqrt_f

```
1 def sqrt_f(n):
2     def f(x):
3         return x*x - n
4     return f
```

number in, function out

```
sqrt_f(5) → function x*x - 5
sqrt_f(6) → function x*x - 6
```

```
f5 = sqrt_f(5); f5(3)                  → 4
sqrt_f(6)(3)                           → 3
newton1(sqrt_f(5), dsqrt_f(5), 1.1)    → 2.236068123793564
```

The notebook's first try, `sqrt_f(x, n)`, failed, as `newton1` calls `f(g0)` with one argument. So `sqrt_f(n)` builds and returns a one-argument function for that `n`; the notes' `derivative(f)` returns one too. How the inner `f` keeps `n` is on the closures sheet.

11 Common mistakes, with square, apply_twice, words = ["fig", "apple", "kiwi"] and nums = [1, 2, 3]

you wrote	the last line Python prints, or what you see; why, and the fix
`print(square)`	`prints <function square at ...>` the bare name is the function; `print(square(4))` prints 16
`apply_twice(square(3), 2)`	`TypeError: 'int' object is not callable` `square(3)` ran first and passed the number 9; pass `square` itself: 16
`sorted(words, key=len())`	`TypeError: len() takes exactly one argument (0 given)` `len()` calls `len` with nothing; `key=len` gives `['fig', 'kiwi', 'apple']`
`m = map(square, nums); print(m[0])`	`TypeError: 'map' object is not subscriptable` an iterator has no positions; `list(map(square, nums))[0]` gives 1
`f = lambda x: return x * x`	`SyntaxError: invalid syntax` a lambda holds one expression, not a statement; drop `return`: `f(3)` gives 9

12 Try it (answers in green)

1. `inc(n)` returns `n + 1`. After `g = inc`, `g(g(g(0)))`?
   `3`: `g` is `inc`, called three times.
2. `list(map(len, ["hi", "hey", "a"]))`?
   `[2, 3, 1]`
3. `list(filter(lambda n: n % 3 == 0, range(9)))`?
   `[0, 3, 6]`
4. `sorted(["pear", "fig", "kiwi"], key=len)`?
   `['fig', 'pear', 'kiwi']`: ties keep order.
5. `max(["kiwi", "banana", "fig"], key=lambda w: w[-1])`?
   `'kiwi'`: its last letter `'i'` is the largest.
6. `apply_twice(lambda s: s + "!", "hi")`?
   `'hi!!'`: the lambda runs twice.

a function is a value · `f` is the function, `f(x)` its result · pass it uncalled to `map`, `filter`, `key=` · `lambda` is a one-line def

Source: the higher-order functions notebook and its handwritten notes (Newton's method, sqrt_f, derivative) and the second functions notebook, nipunbatra.github.io/comp22; apply_twice, do_to_all and the map, filter, key and lambda examples are our own. Every output and error message comes from running the code with Python 3.11.11; memory addresses are shown as "...". Next: the scope sheet.